RNN HIV 5-Fold Statistics Revize

This notebook evaluates a SimpleRNN classifier with stratified 5-fold CV and exports fold-level and summary statistics.

```
!pip -q install rdkit-pypi
```

```
ERROR: Could not find a version that satisfies the requirement rdkit-pypi (from versions: none)
ERROR: No matching distribution found for rdkit-pypi
```

```
import os, random, warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import StratifiedKFold, train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, SimpleRNN, Dense, Dropout, Masking
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping
import tensorflow as tf
from tensorflow import keras

warnings.filterwarnings("ignore")

SEED = 23
MAX_LEN = 32
EMBED_DIM = 64
RNN_UNITS = 64
DROPOUT = 0.40
LR = 5e-4
BATCH_SIZE = 64
EPOCHS = 40
PATIENCE = 8

def set_seed(seed=SEED):
    random.seed(seed)
    np.random.seed(seed)
    tf.random.set_seed(seed)

set_seed(SEED)
```

```
csv_path = keras.utils.get_file(
    "HIV7.csv",
    "https://raw.githubusercontent.com/McahitKutsal/hivcsv/main/HIV7.csv"
)
df = pd.read_csv(csv_path)
df["smiles"] = df["smiles"].astype(str)
df = df[df["smiles"].str.len() < MAX_LEN].reset_index(drop=True)
df = df[["smiles", "HIV_active"]].dropna().reset_index(drop=True)

print(df.shape)
print(df["HIV_active"].value_counts())
df.head()
```

```
(1666, 2)
HIV_active
0    1412
1     254
Name: count, dtype: int64
```

	smiles	HIV_active
0	CNC(=O)Nc1ccncc1	0
1	O=C1NC(O)c2cccc(Cl)c2O1	0
2	O=S1(=O)C=Cc2ccccc2CC1	0
3	C=CCC(=S)NCCc1c[nH]c2ccccc12	0
4	Cc1c2oc(=O)c(C#N)cc2c2ccccn12	1

```
def build_vocab(smiles_series):
    chars = sorted(set("".join(smiles_series.tolist())))
    char_to_int = {c: i + 1 for i, c in enumerate(chars)}  # 0 reserved for padding
    int_to_char = {i: c for c, i in char_to_int.items()}
    return chars, char_to_int, int_to_char
```

```
def vectorize(smiles_series, char_to_int, max_len=MAX_LEN):
    arr = np.zeros((len(smiles_series), max_len), dtype=np.int32)
    for i, s in enumerate(smiles_series.tolist()):
        for j, ch in enumerate(s[:max_len]):
            arr[i, j] = char_to_int.get(ch, 0)
    return arr

def build_rnn_model(vocab_size, max_len=MAX_LEN):
    model = Sequential([
        Embedding(input_dim=vocab_size + 1, output_dim=EMBED_DIM, input_length=max_len, mask_zero=True),
        SimpleRNN(RNN_UNITS, dropout=DROPOUT, recurrent_dropout=0.0),
        Dropout(DROPOUT),
        Dense(64, activation="relu"),
        Dropout(0.20),
        Dense(1, activation="sigmoid")
    ])
    model.compile(
        optimizer=Adam(learning_rate=LR),
        loss="binary_crossentropy",
        metrics=[tf.keras.metrics.BinaryAccuracy(name="accuracy"),
                 tf.keras.metrics.AUC(name="auc")]
    )
    return model

def find_best_threshold_by_f1(y_true, y_prob):
    thresholds = np.arange(0.10, 0.91, 0.02)
    best_thr = 0.50
    best_f1 = -1.0
    for thr in thresholds:
        y_pred = (y_prob >= thr).astype(int)
        score = f1_score(y_true, y_pred, zero_division=0)
        if score > best_f1:
            best_f1 = score
            best_thr = thr
    return float(best_thr), float(best_f1)

def summarize_history(history):
    hist = history.history
    return {
        "loss": hist.get("loss", []),
        "val_loss": hist.get("val_loss", []),
        "accuracy": hist.get("accuracy", hist.get("binary_accuracy", [])),
        "val_accuracy": hist.get("val_accuracy", hist.get("val_binary_accuracy", [])),
        "auc": hist.get("auc", []),
        "val_auc": hist.get("val_auc", [])
    }
```

```
def run_fold(train_df, val_df, test_df, fold_id):
    set_seed(SEED + fold_id)

    _, char_to_int, _ = build_vocab(train_df["smiles"])
    vocab_size = len(char_to_int)

    X_train = vectorize(train_df["smiles"], char_to_int, MAX_LEN)
    X_val   = vectorize(val_df["smiles"], char_to_int, MAX_LEN)
    X_test  = vectorize(test_df["smiles"], char_to_int, MAX_LEN)

    y_train = train_df["HIV_active"].astype(np.float32).values
    y_val   = val_df["HIV_active"].astype(np.float32).values
    y_test  = test_df["HIV_active"].astype(np.float32).values

    model = build_rnn_model(vocab_size, MAX_LEN)

    early_stop = EarlyStopping(
        monitor="val_auc",
        mode="max",
        patience=PATIENCE,
        restore_best_weights=True,
        verbose=0
    )

    history = model.fit(
        X_train, y_train,
        validation_data=(X_val, y_val),
        epochs=EPOCHS,
        batch_size=BATCH_SIZE,
        callbacks=[early_stop],
        verbose=0,
        shuffle=True
    )

    val_probs = model.predict(X_val, verbose=0).ravel()
    best_threshold, best_val_f1 = find_best_threshold_by_f1(y_val, val_probs)

    test_probs = model.predict(X_test, verbose=0).ravel()
    y_pred = (test_probs >= best_threshold).astype(int)

    fold_result = {
```

```
        "fold": fold_id,
        "test_accuracy": accuracy_score(y_test, y_pred),
        "test_precision": precision_score(y_test, y_pred, zero_division=0),
        "test_recall": recall_score(y_test, y_pred, zero_division=0),
        "test_f1": f1_score(y_test, y_pred, zero_division=0),
        "test_roc_auc": roc_auc_score(y_test, test_probs),
        "best_threshold": best_threshold,
        "val_best_f1": best_val_f1,
        "epochs_ran": len(history.history["loss"])
    }

    return fold_result, summarize_history(history)
```

```
all_fold_results = []
all_histories = []

skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)
X_index = np.arange(len(df))
y = df["HIV_active"].values

for fold_id, (train_idx, temp_idx) in enumerate(skf.split(X_index, y), start=1):
    train_df = df.iloc[train_idx].reset_index(drop=True)
    temp_df = df.iloc[temp_idx].reset_index(drop=True)

    val_idx, test_idx = train_test_split(
        np.arange(len(temp_df)),
        test_size=0.5,
        random_state=SEED + fold_id,
        stratify=temp_df["HIV_active"]
    )

    val_df = temp_df.iloc[val_idx].reset_index(drop=True)
    test_df = temp_df.iloc[test_idx].reset_index(drop=True)

    print(f"\n===== Fold {fold_id} =====")
    print("Train:", len(train_df), "Val:", len(val_df), "Test:", len(test_df))

    fold_result, hist = run_fold(train_df, val_df, test_df, fold_id)
    print(fold_result)

    all_fold_results.append(fold_result)
    all_histories.append(hist)

results_df = pd.DataFrame(all_fold_results)
results_df
```

```
===== Fold 1 =====
Train: 1332 Val: 167 Test: 167
{'fold': 1, 'test_accuracy': 0.8323353293413174, 'test_precision': 0.45161290322580644, 'test_recall': 0.56, 'tes

===== Fold 2 =====
Train: 1333 Val: 166 Test: 167
{'fold': 2, 'test_accuracy': 0.8622754491017964, 'test_precision': 0.5714285714285714, 'test_recall': 0.32, 'test

===== Fold 3 =====
Train: 1333 Val: 166 Test: 167
{'fold': 3, 'test_accuracy': 0.8502994011976048, 'test_precision': 0.5263157894736842, 'test_recall': 0.384615384

===== Fold 4 =====
Train: 1333 Val: 166 Test: 167
{'fold': 4, 'test_accuracy': 0.7964071856287425, 'test_precision': 0.36666666666666664, 'test_recall': 0.42307692

===== Fold 5 =====
Train: 1333 Val: 166 Test: 167
{'fold': 5, 'test_accuracy': 0.3473053892215569, 'test_precision': 0.16260162601626016, 'test_recall': 0.76923076
```

	fold	test_accuracy	test_precision	test_recall	test_f1	test_roc_auc	best_threshold	val_best_f1	epochs_ran
0	1	0.832335	0.451613	0.560000	0.500000	0.746197	0.16	0.440678	35
1	2	0.862275	0.571429	0.320000	0.410256	0.776338	0.24	0.350000	30
2	3	0.850299	0.526316	0.384615	0.444444	0.745499	0.24	0.577778	40
3	4	0.796407	0.366667	0.423077	0.392857	0.738953	0.24	0.521739	17
4	5	0.347305	0.162602	0.769231	0.268456	0.582651	0.16	0.287500	11

```
summary_rows = []
for metric in ["test_accuracy", "test_precision", "test_recall", "test_f1", "test_roc_auc"]:
    mean = results_df[metric].mean()
    std = results_df[metric].std(ddof=1)
    var = results_df[metric].var(ddof=1)
    summary_rows.append({
```

```
        "Metric": metric.replace("test_", "").upper(),
        "Mean": mean,
        "Std": std,
        "Variance": var,
        "Formatted": f"{mean:.3f} ± {std:.3f}"
    })

summary_df = pd.DataFrame(summary_rows)
summary_df
```

	Metric	Mean	Std	Variance	Formatted
0	ACCURACY	0.737725	0.219663	0.048252	0.738 ± 0.220
1	PRECISION	0.415725	0.161431	0.026060	0.416 ± 0.161
2	RECALL	0.491385	0.178427	0.031836	0.491 ± 0.178
3	F1	0.403203	0.085709	0.007346	0.403 ± 0.086
4	ROC_AUC	0.717928	0.076995	0.005928	0.718 ± 0.077

```
final_table = pd.DataFrame([{
    "Model": "RNN",
    "Accuracy": summary_df.loc[summary_df["Metric"] == "ACCURACY", "Formatted"].iloc[0],
    "Precision": summary_df.loc[summary_df["Metric"] == "PRECISION", "Formatted"].iloc[0],
    "Recall": summary_df.loc[summary_df["Metric"] == "RECALL", "Formatted"].iloc[0],
    "F1": summary_df.loc[summary_df["Metric"] == "F1", "Formatted"].iloc[0],
    "ROC-AUC": summary_df.loc[summary_df["Metric"] == "ROC_AUC", "Formatted"].iloc[0],
}])

results_df.to_csv("RNN_fold_results.csv", index=False)
summary_df.to_csv("RNN_summary_results.csv", index=False)
final_table.to_csv("RNN_final_table.csv", index=False)

final_table
```

	Model	Accuracy	Precision	Recall	F1	ROC-AUC
0	RNN	0.738 ± 0.220	0.416 ± 0.161	0.491 ± 0.178	0.403 ± 0.086	0.718 ± 0.077

```
max_len_hist = max(len(h["loss"]) for h in all_histories)

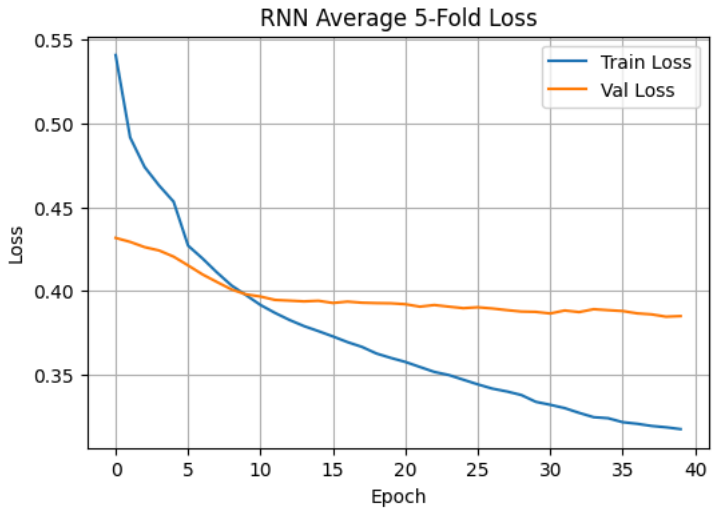
def pad_hist(hist_list, key):
    arr = []
    for h in hist_list:
        vals = h.get(key, [])
        if len(vals) == 0:
            vals = [0.0] * max_len_hist
        elif len(vals) < max_len_hist:
            vals = vals + [vals[-1]] * (max_len_hist - len(vals))
        arr.append(vals)
    return np.array(arr)

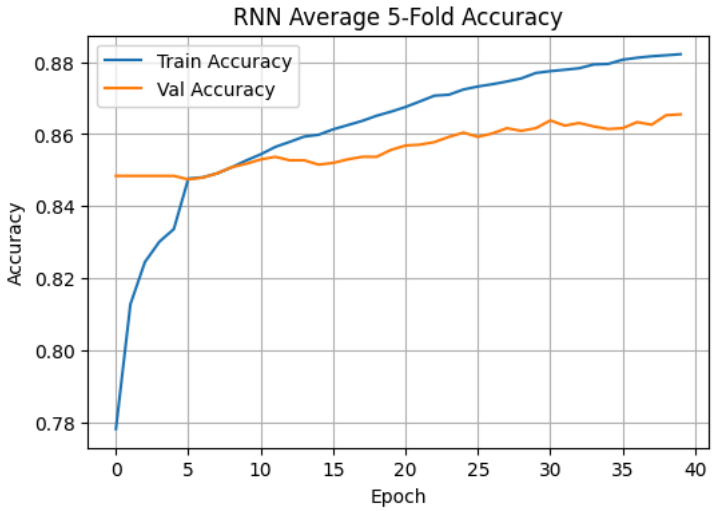
def smooth_curve(values, window=5):
    smoothed = []
    for i in range(len(values)):
        start = max(0, i - window + 1)
        smoothed.append(np.mean(values[start:i+1]))
    return smoothed

loss_arr = pad_hist(all_histories, "loss")
val_loss_arr = pad_hist(all_histories, "val_loss")
acc_arr = pad_hist(all_histories, "accuracy")
val_acc_arr = pad_hist(all_histories, "val_accuracy")

plt.figure(figsize=(6,4))
plt.plot(smooth_curve(loss_arr.mean(axis=0), window=5), label="Train Loss")
plt.plot(smooth_curve(val_loss_arr.mean(axis=0), window=5), label="Val Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("RNN Average 5-Fold Loss")
plt.legend()
plt.grid(True)
plt.show()

plt.figure(figsize=(6,4))
plt.plot(smooth_curve(acc_arr.mean(axis=0), window=5), label="Train Accuracy")
plt.plot(smooth_curve(val_acc_arr.mean(axis=0), window=5), label="Val Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("RNN Average 5-Fold Accuracy")
plt.legend()
plt.grid(True)
plt.show()
```


RNN Average 5-Fold Loss
Train Loss
Val Loss
Loss
Epoch
0.55
0.50
0.45
0.40
0.35
0
5
10
15
20
25
30
35
40


RNN Average 5-Fold Accuracy
Train Accuracy
Val Accuracy
Accuracy
Epoch
0.88
0.86
0.84
0.82
0.80
0.78
0
5
10
15
20
25
30
35
40